3. laboratorijska vježba

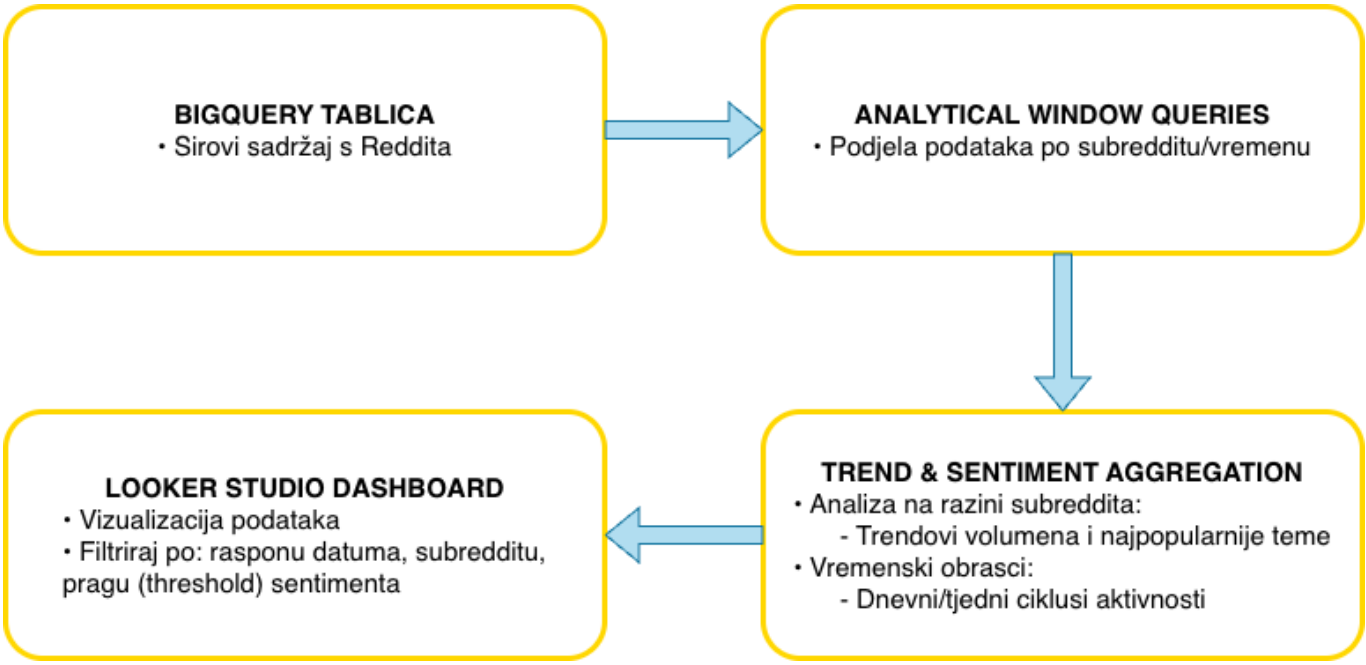
- Uvod
- BigQuery Analitika
  - Windowing Queries
  - Aggregations
- dbt (Data Build Tool)
- Data Quality
- Looker Studio
- 1. Zadatak
- 2. Zadatak
- VAŽNE NAPOMENE

Uvod

U prethodnoj vježbi implementirali smo data ingestion pipeline: producer šalje poruke u Pub/Sub, consumer ih pohranjuje u GCS u Parquet formatu i sinka u BigQuery, dok nevalidne poruke završavaju u dead-letter topic-u.

U ovoj vježbi nadograđujemo pipeline dodavanjem analitike i vizualizacije podataka. Cilj je izvući relevantne informacije iz podataka, analizirati trendove po sentimentu, te vizualizirati rezultate u Looker Studio-u.

Pregled arhitekture



Cilj ove laboratorijske vježbe je:

- Pripremiti i transformirati podatke u BigQuery-u koristeći dbt, koristeći medallion arhitekturu.
- Primijeniti window funkcije i agregacije u SQL-u unutar dbt modela za analizu trendova i broja objava po sentimentu.
- Validirati podatke pomoću Great Expectations kako bi se osigurala kvaliteta i integritet prije vizualizacije.

- Kreirati vizualizacije i dashboard u Looker Studio-u

BigQuery Analitika

Windowing Queries

Window functions omogućuju izvođenje analiza nad dijelovima podataka (window), bez potrebe za grupiranjem cijelog skupa podataka.

Primjeri uporabe:

```
SELECT
  title,
  created_utc,
  ROW_NUMBER() OVER (PARTITION BY created_utc ORDER BY created_utc DESC)
AS row_num
FROM
  `my-project-id.reddit_pipeline.reddit_messages`;
```

Svrha u pipeline-u:

- Omogućuje detekciju novih objava i trendova u stvarnom vremenu.
- Pomaže u kreiranju rangiranja ili rolling analiza.

Aggregations

Agregacije omogućuju grupiranje podataka i izračun statistika.

Primjeri uporabe:

```
SELECT
  EXTRACT(DATE FROM created_utc) AS date,
  EXTRACT(HOUR FROM created_utc) AS hour,
  COUNT(*) AS post_count
FROM
  `project_id.reddit_pipeline.reddit_messages`
GROUP BY
  date, hour
ORDER BY
  date, hour;

SELECT
  sentiment,
  COUNT(*) AS sentiment_count
FROM
  `project_id.reddit_pipeline.reddit_messages`
GROUP BY
  sentiment
```

```
ORDER BY
  sentiment;
```

Svrha u pipeline-u:

- Praćenje distribucije sentimenta (pozitivno, negativno, neutralno).
- Osnova za vizualizacije i dashboard.

dbt (Data Build Tool)

Dbt (Data Build Tool) je alat za transformaciju podataka koji koristi SQL. Njegova osnovna svrha je strukturirati i automatizirati obradu podataka, čime pipeline postaje modularan, lako reproducibilan i jednostavniji za održavanje.

Glavne prednosti su:

- Strukturiranje pipeline-a u slojeve: omogućuje jasno definiranje slojeva transformacija (bronze, silver, gold) i njihovih međusobnih ovisnosti.
- Praćenje ovisnosti i vizualizacija: automatski generira DAG (Directed Acyclic Graph) koji pokazuje kako modeli i tablice ovise jedni o drugima.
- Testiranje podataka: omogućuje dodavanje automatskih testova (npr. not null, unique, accepted values) kako bi se osigurala kvaliteta podataka.
- Laka reproducibilnost i debug: transformacije se mogu pokrenuti i testirati lokalno ili u cloudu, što olakšava pronalaženje i ispravljanje grešaka.
- Integracija s alatima za data quality: poput Great Expectations, za dodatnu provjeru i validaciju podataka prije agregacija i vizualizacija.

Bez dbt-a: Sve transformacije morale bi se raditi ručno, teško bi se pratila ovisnost među tablicama, a testiranje i dokumentacija bili bi ručni i skloniji pogreškama.

U kontekstu ove vježbe: Dbt omogućuje da srebrni i zlatni sloj podataka u BigQuery-u bude modularno definiran, testiran i reproducibilan. Možete jasno vidjeti redoslijed transformacija, debugirati pojedine modele, te lako dodavati testove i kvalitetne transformacije za dashboard u Looker Studio.

Ključni koncepti za rad:

- Models (.sql): SQL datoteke koje definiraju transformacije.
- Ref funkcija ({{ ref('model_name') }}): ključna funkcija koja povezuje modele. Omogućuje dbt-u da zna kojim redoslijedom pokretati modele i automatski crta graf ovisnosti (Lineage Graph).
- Documentation (.yml): Datoteke u kojima opisujete modele i stupce te definirate testove kvalitete podataka. One su ključne za dbt docs.

Data Quality

Great Expectations (GE) je framework za provjeru i dokumentaciju kvalitete podataka. Omogućuje definiranje “expectations” - pravila i uvjeta koje podaci moraju zadovoljavati i automatski provjerava jesu li podaci u skladu s tim pravilima.

Zašto ga koristimo:

- Provjerava integritet i kvalitetu podataka prije nego što se pohrane u zlatni sloj ili koriste za vizualizacije.
- Omogućuje automatsko generiranje izvještaja i dokumentacije o kvaliteti podataka.
- Integrira se s dbt-om tako da testovi kvalitete mogu biti dio pipeline-a i reproducibilno se pokreću za svaki update podataka.
- Primjeri očekivanja podataka (Expectations):
  - Provjera postoje li nove kolone
  - Polje id mora biti jedinstveno (unique)
  - Duljina naslova posta ne smije prelaziti određeni broj znakova

Integracija u pipeline i dbt:

- Bronze sloj: pasivno praćenje podataka - provjera strukture, volumena i jedinstvenosti ID-eva; upozorenja za povećane NULL vrijednosti...
- Silver sloj: provjera čišćenih i standardiziranih podataka - kritične kolone bez NULL, vrijednosti iz dozvoljenog skupa, duljina naslova unutar granica...

Instalacija i osnovno korištenje dbt-a i Great Expectations

1. Instalacija dbt-a

- dbt se može instalirati lokalno putem pip-a

```
pip install dbt-bigquery
```

2. Osnovna konfiguracija dbt-a

- Kreirajte novi dbt projekt:

```
dbt init reddit_project
```

- Popunite potrebna polja pri inicijalizaciji projekta (bit će ponuđeno u terminalu)
- Možete se autentificirati s oauth ili pomoću servisnog računa što zahtjeva dohvaćanje JSON ključa, preporučamo oauth jer je jednostavnije
- Druga opcija za konfiguraciju je putem profiles.yml (unutar .dbt na vašem računalu):

```
reddit_project:
  target: dev
  outputs:
    dev:
      type: bigquery
      method: service-account
      project: <YOUR_PROJECT_ID>
      dataset: reddit_pipeline
```

```
      keyfile: <PATH_TO_SERVICE_ACCOUNT_JSON>
      threads: 4
      timeout_seconds: 300
```

- Pod keyfile navedite putanju do vašeg Service Account JSON-a koji ste preuzeli s Google Clouda.
- Organizacija projekta:
  - models/bronze/ - sirovi podaci
  - models/silver/ - očišćeni i standardizirani podaci
  - models/gold/ - agregirani podaci za izvještavanje
- Pokretanje transformacija:

```
dbt run
```

- Pokretanjem run će se kreirati nove tablice u vašem dataset-u primjenom SQL upita definiranog dbt modelom (dbt_projekt/models)
- Testiranje i validacija modela:

```
dbt test
```

- Ili pokretanje, testiranje i validacija modela u jednom:

```
dbt build
```

- Preporuča se korištenje naredbe build za kraj jer ona zaustavlja proces ako testovi padnu, čime se sprječava da "loši" podaci dolaze do vizualizacija
- Vizualizacija DAG-a (Lineage Graph) - naredbe u nastavku generiraju web sučelje u kojem možete pregledati grafički prikaz vašeg pipeline-a i detalje o svim modelima.

```
dbt docs generate
dbt docs serve
```

- Iako će dbt generirati Lineage Graph (DAG) automatski na temelju vašeg SQL koda, preporučuje se kreiranje .yml datoteka unutar models/ foldera. Te datoteke omogućuju dodavanje tekstualnih opisa stupaca, poslovnih pravila i informacija o testovima (poput unique ili not_null), čime dokumentacija postaje potpun i razumljiv tehnički priručnik vašeg sustava.

3. Instalacija i konfiguracija Great Expectations

```
pip install great_expectations==0.15.42
```

- Inicijalizacija projekta - prvo se pozicionirajte u folder vašeg dbt projekta:

```
great_expectations init
```

- Potrebno je kreirati datasource:
  - pokretanjem naredbe u terminalu će vam se prikazati izbornik - odaberite opciju 2. Relational database (SQL), zatim 5. Bigquery čime će se u browseru automatski otvoriti Jupyter bilježnica pomoću koje ćete definirati svoj izvor podataka
  - preuzmite ključ servisnog računa (kliknite na svoj service account te pod Keys generirajte i preuzmite ključ), unesite podatke izvora te pokrenite ćelije

```
great_expectations datasource new
```

```
# The SQLAlchemy url/connection string for the BigQuery connection
# (reference: https://github.com/googleapis/python-bigquery-
sqlalchemy#connection-string-parameters)"""
connection_string = "bigquery://id-projekta/ime-dataseta?
key_path=/put/do/kljuca/key.json"

schema_name = "reddit_dataset"  # or dataset name
table_name = "my_first_dbt_model" # ime vašeg dbt modela kojeg ste
prethodno kreirali
```

- Kreirajte “expectation suite” - u terminalu će vam se pokrenuti wizard za kreiranje, odaberite prvo opciju 2. Interactively, with a sample Batch of data te zatim konektor na vaš izvor 2. default_inferred_data_connector_name, na kraju data asset odnosno dbt model koji želite koristiti
- "Očekivanja" za vaš dbt model definirajte unutar Jupyter bilježnice koja će se automatski otvoriti

```
great_expectations suite new
```

- Checkpoint je konfiguracija koja povezuje Expectation Suite i izvor podataka (datasource). Kad ga pokrenete, provjerava podatke prema pravilima iz suite i generira izvještaj o tome što prolazi, a što ne
- Pokretanje provjere podataka (checkpoint):

```
great_expectations checkpoint run <checkpoint_name>
```

- GE automatski kreira HTML izvještaje s rezultatima testove

Looker Studio

Looker Studio (ranije Google Data Studio) omogućuje kreiranje vizualnih dashboarda na temelju podataka iz BigQuery. Može se spojiti na BigQuery, relacijske baze (MySQL, PostgreSQL), cloud storage (GCS, Google Sheets), datoteke (CSV, Excel) pa čak i web servise poput Google Analytics. Omogućuje brzo vizualiziranje podataka, kreiranje filtriranih i agregiranih prikaza, te dijeljenje dashboarda s timom. U ovoj vježbi koristimo BigQuery kao izvor, ali princip vizualizacije isti je i za druge tipove izvora podataka.

Postupak integracije:

- Otvorite Looker Studio
- Kreirajte novi report → Connect to BigQuery.
- Odaberite projekt i dataset (`ime_dataseta.ime_tablice`).
- Definirajte vizualizacije
- Postavite filtere za odabrane stavke , npr. vrijeme.
- Izvoz dashboarda ili dijeljenje s timom.

> **Napomena:** Prilikom konfiguracije Looker Studio, koristite BigQuery Live Connection ili Extract, ovisno želite li da dashboard uvijek čita svježe podatke ili snapshot.

1. Zadatak

Tema ovog zadatka je implementacija cjelovitog pipeline-a za analizu podataka s Reddita korištenjem medaljonske arhitekture (medallion architecture) u BigQuery-u. Cilj je strukturirati podatke u slojevima: brončani, srebrni i zlatni, te izvesti agregacije za analizu trendova i sentimenta koji će se koristiti za vizualizaciju.

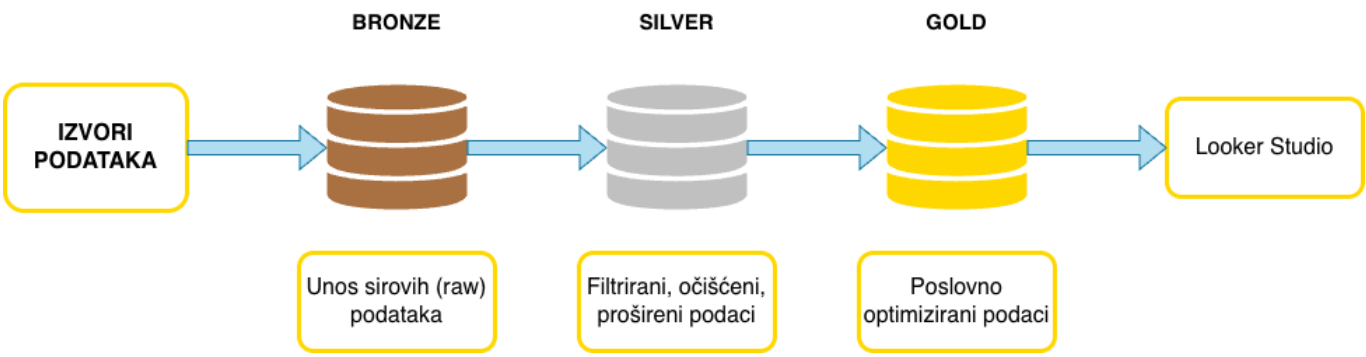
Kontekst

Potrebno je analizirati podatke iz tablice `reddit_messages` (naziv BigQuery tablice u koju ste spremili podatke iz Pub/Sub-a u prošloj laboratorijskoj vježbi) kroz sljedeće faze:

1. Priprema i čišćenje podataka - uklanjanje duplikata, popunjavanje NULL vrijednosti, standardizacija formata.
2. Analiza trendova i sentimenta - broj objava po vremenskim intervalima, klasifikacija sentimenta, redoslijed objava.
3. Agregacija za izvještavanje - kreiranje poslovno relevantnih metrika za dashboarde.

Koncept medaljonske arhitekture:

- Brončani sloj (bronze layer): Sirovi, neoblikovani podaci (raw data) - izvorna tablica `reddit_messages`.
- Srebrni sloj (silver layer): Očišćeni i oblikovani podaci - uklanjanje duplikata, popunjavanje NULL vrijednosti, standardizacija formata.
- Zlatni sloj (gold layer): Agregirani i poslovno relevantni podaci - podaci optimizirani za izvještavanje i dashboarde (npr. dnevni trendovi, sentiment analiza po kategorijama).



Koraci za implementaciju:

1. Kreiranje brončanog sloja

- Tablica `reddit_dataset.reddit_messages` (BigQuery tablica u koju ste pohranili sirove podatke iz Pub/Sub-a u prošloj laboratorijskoj vježbi) služi kao baza za brončani sloj
- Napravite dbt model `models/bronze/reddit_messages_bronze.sql`

```
{{ config(materialized='view') }}

SELECT *
FROM `id_projekta.ime_bigquery_dataseta.ime_bigquery_tablice`
```

- Druga opcija je definirati tablicu kao source, ali u ovom primjeru to ne koristimo kako bi Great Expectations framework mogao direktno raditi na dbt modelu.
- Primjer - kreirajte datoteku models/sources.yml:

```
version: 2
sources:
  - name: reddit_raw_data
    dataset: reddit_pipeline
    tables:
      - name: reddit_messages
```

Definicija Great Expectations pravila

- Provedite postupak kreiranja Great Expectations suite-a prema uputama (unutar Jupyter bilježnica koje se generiraju pomoću navedeih komandi)

- Na brončanom sloju GE provjere služe za monitoring i upozorenja (npr. schema drift, volumen, udio NULL vrijednosti, duplikati) - podaci se ne mijenjaju

    - Provjera schema drift - je li se dodala ili maknula kolona.
    - Provjera volumena podataka - je li je dodan redak.
    - Praćenje jedinstvenosti id.

- Primjer definicije:

```
validator.expect_column_to_exist("id")
validator.expect_column_values_to_be_unique("id")
validator.expect_table_row_count_to_be_between(min_value=1)
validator.expect_column_values_to_not_be_null("created_utc", mostly=0.8)
# do 20% NULL dozvoljeno
```

- Pokrenite provjere prije kreiranja silver sloja kako bi identificirali promjene:

```
great_expectations checkpoint run <checkpoint_name>
```

Particioniranje

Particioniranje: Fizičko dijeljenje tablice (npr. po danima). BigQuery čita samo particije koje su unutar WHERE, što drastično reže troškove (npr. skeniraš 10 MB umjesto 10 GB).

- U dbt modelima (silver i gold) obavezno definirajte particioniranje po datumu i klasterizaciju kako biste smanjili troškove upita u BigQueryju
- Konfiguracija u dbt-u: Na vrh .sql datoteke dodajte config blok (npr.):

```
{{ config(
    materialized='table',
    partition_by={
        "field": "created_utc",
        "data_type": "timestamp",
        "granularity": "day"
    }
) }}
```

2. Kreiranje srebrnog sloja

Cilj ovog modela je transformacija sirovih podataka iz Bronze sloja u očišćenu tablicu spremnu za analizu. Potrebno je implementirati deduplikaciju, čišćenje teksta te osnovnu logiku klasifikacije.

- Napravite dbt model u `models/silver/reddit_messages_silver.sql` koji:
  - Uklanja duplikate: Koristi `ROW_NUMBER()` preko `id` polja, sortirano prema najnovijem vremenu objave (`created_utc`).
  - Standardizira tekst: Čisti `title` od specijalnih znakova i višestrukih razmaka koristeći regex, uz obavezni `TRIM`.
  - Popunjava prazna polja: Koristi `COALESCE` za `score` i `num_comments` (postavlja na 0 ako su `NULL`).
  - Dodaje poslovnu logiku: Klasificira objave u kategorije sentimenta (`HIGH_ENGAGEMENT`, `POSITIVE`, `NEGATIVE`, `NEUTRAL`) na temelju angažmana i bodova.
  - Optimizira pohranu: Tablica mora biti particionirana po danu koristeći `created_utc`.

```sql
-- models/silver/reddit_messages_silver.sql
{{ config(
    materialized='table',
    partition_by={
        "field": "created_utc",
        "data_type": "timestamp",
        "granularity": "day"
    }
) }}

WITH deduplicated AS (
    SELECT
        *,
        -- TODO: Implementirati ROW_NUMBER() za uklanjanje duplikata po
'id' koloni
        -- Poredati po 'created_utc' silazno
        ROW_NUMBER() OVER (...) AS rn
    FROM {{ ref('reddit_messages') }}
)

SELECT
    id,
    -- TODO: Očistiti 'title' (TRIM, uklanjanje specijalnih znakova
regexom i zamjena višestrukih razmaka jednim)
    -- REGEXP_REPLACE(...) AS title_clean,
    author,
    -- TODO: Popuniti NULL vrijednosti u 'score' i 'num_comments'
(koristiti 0 kao default)
    -- COALESCE(...) AS score_filled,
    url,
    selftext,
    created_utc,
    -- TODO: Implementirati sentiment logiku koristeći CASE statement:
    -- 1. score >= 50 i num_comments >= 10 -> 'HIGH_ENGAGEMENT'
    -- 2. score >= 20 -> 'POSITIVE'
    -- 3. score < 0 -> 'NEGATIVE'
    -- 4. Sve ostalo -> 'NEUTRAL'
    CASE
        -- Dopuniti logiku ovdje
    END AS sentiment
FROM deduplicated
WHERE rn = 1 -- Zadržavamo samo unikatne zapise
  -- TODO: Dodati uvjet da 'title' nije NULL i da njegova duljina nakon
TRIM-a nije 0
```

- kreirajte i models/silver/reddit_messages_silver.yml. datoteku koja služi za opisivanje podataka i automatsko generiranje dbt dokumentacije (nadopunite ovaj primjer po potrebi):

```yaml
version: 2

models:
```

```
  - name: reddit_messages_silver
    description: "Srebrni sloj s očišćenim podacima iz Reddita. Sadrži
standardizirane naslove i sentiment."
    columns:
      - name: id
        description: "Jedinstveni identifikator objave."
        tests:
          - unique
          - not_null
      - name: sentiment
        description: "Kategorizirani sentiment (POSITIVE, NEGATIVE,
NEUTRAL)."
```

Definicija Great Expectations pravila

- Kreirajte GE expectation suite za silver sloj (unutar Jupyter bilježnice):
  - `id` mora biti jedinstven (unique)
  - Provjera jesu li uklonjeni svi NULL iz kritičnih kolona (`sentiment`)
  - Duljina naslova ne smije prelaziti određeni broj znakova
- Primjer definicije:

```
validator.expect_column_values_to_be_unique("id")
validator.expect_column_values_to_not_be_null("sentiment")
validator.expect_column_value_lengths_to_be_between("title",
max_value=100)
```

- Pokrenite provjere prije kreiranja gold sloja:

```
great_expectations checkpoint run <checkpoint_name>
```

3. Kreiranje zlatnog sloja

Napravite dbt model u `models/gold/reddit_messages_gold.sql` koji agregira podatke iz Silver sloja kako bi se dobili ključni poslovni uvidi. Vaš zadatak je popuniti SQL logiku na mjestima označenim s `TODO`. Zadaci modela:

- Agregirati dnevnu statistiku po sentimentu.
- Izračunati pomični zbroj (rolling sum) objava za zadnjih 7 dana.
- Izračunati postotnu promjenu broja objava u odnosu na prethodni dan (Day-over-Day).
- Osigurati particioniranje tablice po datumu radi optimizacije troškova.

```
{{ config(
    materialized='table',
    partition_by={
```

```
        "field": "report_date",
        "data_type": "date"
    }
) }}

WITH daily_aggregates AS (
    SELECT
        -- TODO: Izvucite datum iz created_utc i nazovite ga report_date

        sentiment,
        COUNT(*) AS post_count,
        AVG(score_filled) AS avg_score,
        SUM(score_filled) AS total_score,
        COUNT(DISTINCT author) AS unique_authors
    FROM {{ ref('reddit_messages_silver') }}
    -- TODO: Grupirajte podatke po report_date i sentimentu

),

window_calculations AS (
    SELECT
        *,
        -- TODO: Izračunajte pomični zbroj (SUM) post_count-a kroz zadnjih
7 dana
        -- Koristite PARTITION BY sentiment i ORDER BY report_date
        -- Definirajte prozor: ROWS BETWEEN 6 PRECEDING AND CURRENT ROW

        -- TODO: Dohvatite vrijednost post_count-a od prethodnog dana
koristeći LAG funkciju
        AS prev_day_posts
    FROM daily_aggregates
)

SELECT
    *,
    -- TODO: Izračunajte postotnu promjenu: ((trenutno - prethodno) /
prethodno) * 100
    -- Koristite CASE WHEN kako biste izbjegli dijeljenje s nulom (ako je
prev_day_posts > 0)
    AS day_over_day_change_pct
FROM window_calculations
```

Ovaj model koristi samo validne i provjerene podatke iz silver sloja, što osigurava da dashboard u Looker Studio prikazuje točne i pouzdane informacije.

- U models/gold/reddit_messages_gold.yml definirajte opise za poslovne metrike koje će se koristiti u Looker Studiju (nadopunite primjer po potrebi):

```
version: 2

models:
```

```
    - name: reddit_messages_gold
      description: "Zlatni sloj optimiziran za Looker Studio dashboard.
  Sadrži agregirane trendove i dnevne metrike."
      columns:
        - name: report_date
          description: "Datum za koji su podaci agregirani."
        - name: post_count
          description: "Ukupan broj objava po sentimentu na taj dan."
```

- Generirajte Lineage Graph pomoću dbt docs - ako u Lineage Graphu vidite nepovezane modele (otoke), provjerite jeste li svugdje koristili ref funkcije.

Tehnički zahtjevi:

- Primijenite dbt modele na podatke
- Definirajte Great Expectations pravila na bronze i silver sloj
- Definirajte odgovarajuće particije i klastere za performanse (npr. particije po datumu za srebrni i zlatni sloj).
- Generirajte dokumentaciju

2. Zadatak

Tema ovog zadatka je vizualizacija podataka u Looker Studio-u koristeći zlatni sloj podataka (`reddit_messages_gold`) koji je generiran dbt modelima u BigQuery-u.

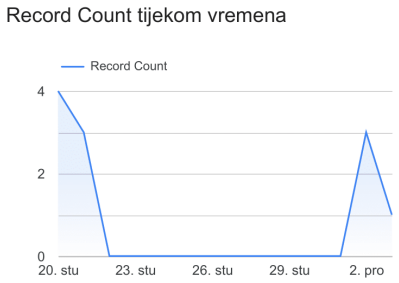
- Otvorite Looker Studio i kreirajte novi report.
- Povežite BigQuery dataset `reddit_pipeline.reddit_messages_gold` (ime vaše gold tablice).
- Kreirajte dashboard koji uključuje:
  - Trend sentimenta kroz vrijeme (linijski graf)
- Definirajte vizualizacije
  - Line chart: Broj objava po vremenu (trend)
- Dodajte filtere
  - Filtriranje po vremenskom rasponu
- Spremite i provjerite dashboard
  - Provjerite da se podaci ažuriraju u stvarnom vremenu ili prema odabiru Live/Extract konekcije.

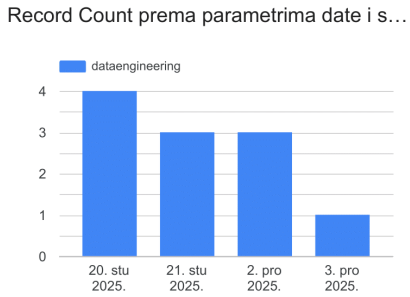
Optimizacija upita: Budući da smo u dbt-u postavili particioniranje po datumu, u Looker Studiju obavezno dodajte "Date Range Control". To će prisiliti Looker da šalje optimizirane upite BigQuery-u, smanjujući troškove i ubrzavajući prikaz.

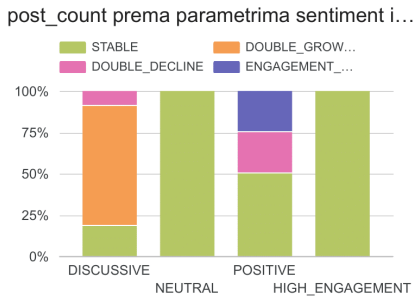
Live Connection: Koristite Live Connection kako bi vaš dashboard uvijek prikazivao najnovije stanje iz Gold tablice bez potrebe za ručnim osvježavanjem podataka.

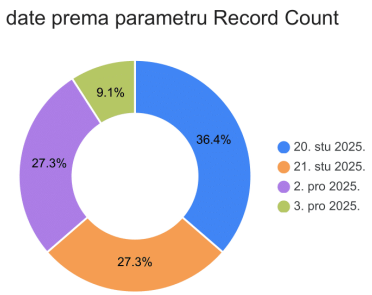
> **Napomena**: Gold sloj (reddit_messages_gold) uvijek sadrži validirane i agregirane podatke koji se koriste za dashboard. Sve transformacije, agregacije i rangiranja se odvijaju u dbt modelima, tako da Looker Studio ne treba izvršavati dodatni SQL za window funkcije ili agregacije.

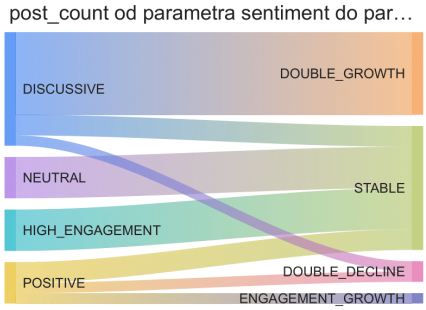
Ispod su primjeri nekih od mogućih grafova koji se mogu prikazati na temelju poslovno optimiziranih podataka zlatnog sloja za Reddit temu "r/dataengineering".

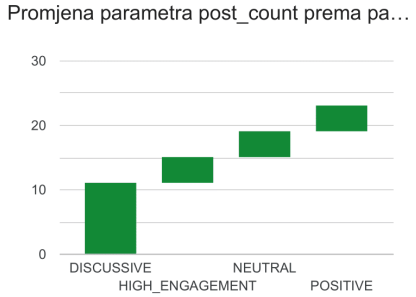












VAŽNE NAPOMENE

U ovoj laboratorijskoj vježbi smo pojednostavili posljednji dio pipeline-a kako bi mogli praktično razumjeti i isprobati ključne tehnologije poput dbt-a, Great Expectations i Looker Studija, bez potrebe za stvarnim periodičnim pokretanjem svih komponenti.

Producer i dohvaćanje podataka

Za potrebe vježbe, producer je simuliran kao cloud job koji dohvaća podatke samo jednom i puni BigQuery tablicu. U praksi bi producer stalno slušao nove podatke na Pub/Sub-u ili nekom drugom messaging sustavu i kontinuirano slao nove poruke u pipeline, čime bi pipeline bio potpuno real-time ili near real-time.

dbt modeli i periodičko ažuriranje

- U ovoj vježbi dbt modeli su pokrenuti ručno, jednom, kako bi se izradili srebrni i zlatni sloj podataka.
- U produkciji bi dbt modeli bili schedulirani (npr. putem Airflow-a, Cloud Composer-a ili dbt Cloud Scheduler-a) i automatski pokretani periodično ili po događaju, što omogućuje stalno ažuriranje silver i gold slojeva kad novi podaci stignu.
- Tako bi svi agregati, rankinzi i trendovi u gold sloju bili uvijek svježi i spremni za vizualizaciju u Looker Studio-u.
- Dodatno, u profesionalnom okruženju, cijeli proces transformacija i testova mogao bi se pokretati kroz CI/CD pipeline, čime bi se osigurala automatizacija, verzioniranje modela i konzistentnost između razvoja i produkcije.

Great Expectations i kvalitetna kontrola podataka

- GE je integriran s dbt modelima kako bi se provjerila kvaliteta podataka prije nego što se stignu koristiti u vizualizacijama.
- U produkciji bi se checkpointi i testovi pokretali automatski nakon svakog batch update-a, čime bi se spriječilo da loši ili nepotpuni podaci dospiju u gold sloj.

Looker Studio vizualizacije

- Vizualizacije su kreirane pod pretpostavkom da se podaci periodično ažuriraju.
- U produkciji bi Live Connection ili Extract bile povezane s automatski ažuriranim gold slojem, čime bi dashboard uvijek prikazivao najnovije stanje podataka.
- Filteri, date range kontrole i agregacije u Looker Studio-u rade optimalno jer su svi podaci prethodno agregirani i validirani u gold sloju, što smanjuje opterećenje baze i ubrzava prikaz.

Dobre prakse i preporuke

- Medallion arhitektura: korištenje bronze, silver i gold slojeva omogućuje modularnost, bolju kontrolu kvalitete i jednostavno debuggiranje.
- Automatizacija i scheduliranje: sve transformacije i testovi bi u produkciji trebali biti automatizirani i reproducibilni.
- Kontrola kvalitete: definiranje expectation suite-ova i testova u dbt-u osigurava da samo validni podaci dolaze do vizualizacija.
- Particioniranje: koristi se za optimizaciju performansi i smanjenje troškova upita u BigQuery-u.
- Reproducibilnost i dokumentacija: dbt docs i GE izvještaji olakšavaju praćenje promjena, testiranje i komunikaciju između članova tima.

Ukratko, u ovoj vježbi smo napravili statičnu verziju pipeline-a kako bismo se fokusirali na upoznavanje tehnologija i principa, ali sve komponente su postavljene tako da se lako mogu nadograditi u produkcijsko, periodički ažurirano okruženje.